



Universidad de Carabobo

Profesora: Mirella Herrera
Estudiante: Jesus Hernandez,
CI: 31456568

Mecanismos de Exclusión mutua

Junio, 2026

Sistemas Operativos - Asignacion 3.
Dpto Computación, Facyt

Exclusión mutua

Es un mecanismo que garantiza que un solo proceso o hilo puede acceder a un recurso compartido en un momento determinado.

ejemplo

- Se tiene una cuenta bancaria con 1000bs
- Dos procesos (PC y Telefono) intentan sacar dinero al mismo tiempo.
- Ambos leen el saldo: 1000bs
 - El PC resta 500bs → Saldo ahora es 500bs
 - Pero el telefono resta 300bs → Saldo ahora es 700bs

¡De 1000bs se sacaron 800bs pero solo se restaron 300!



Soluciones de Software



Estas soluciones dependen de algoritmos lógicos sin necesidad de soporte especial del hardware (mas allá de la exclusión mutua elemental en el acceso a memoria):

Librería Mutex

uso en programas

Librería auxiliar para creación y manejo de hilos, ejecutarlos de forma concurrente y esperar a que terminen.

Esto es necesario para poder mostrar con rigurosidad la efectividad de los algoritmos. De no usarlos los programas se ejecutan secuencialmente.

```
int main() {  
    ThreadPair procesos = crear_procesos(proceso_0, proceso_1);  
    esperar_procesos(procesos);  
    return 0;  
}
```

implementación

```
#include "mutex.h"  
#include <stdio.h>  
  
ThreadPair crear_procesos(void* (*proceso_0)(void*),  
                          void* (*proceso_1)(void*)) {  
    ThreadPair pair;  
  
    pthread_create(&pair.thread0, NULL, proceso_0, NULL);  
    pthread_create(&pair.thread1, NULL, proceso_1, NULL);  
  
    return pair;  
}  
  
void esperar_procesos(ThreadPair pair) {  
    pthread_join(pair.thread0, NULL);  
    pthread_join(pair.thread1, NULL);  
}
```

definición

```
#ifndef MUTEX_H  
#define MUTEX_H  
  
#include <pthread.h>  
  
// Estructura para guardar info de los threads  
typedef struct {  
    pthread_t thread0;  
    pthread_t thread1;  
} ThreadPair;  
  
// Crear y ejecutar dos procesos concurrentemente  
ThreadPair crear_procesos(void* (*proceso_0)(void*),  
                          void* (*proceso_1)(void*));  
  
// Esperar a que terminen  
void esperar_procesos(ThreadPair pair);  
  
#endif
```


Algoritmo de Dekker V1 - Alternancia Simple

Idea: Usa una variable *turn* para decidir qué proceso puede entrar a la sección crítica.

Si no es su turno, debe esperar.

- Si entra P0, espera P1.
- Si entra P1, espera P0.

Cabecera

```
#include <stdio.h>
#include "mutex.h"

// VARIABLES COMPARTIDAS
int turnoHilo = 0;
int totalHilos = 2;
int estadoHilo[2] = {0, 0};
int saldo = 1000;
```

Problema

La alternancia es forzada:

Un proceso **no puede entrar dos veces seguidas**, aunque el otro no quiera usar el recurso.

Si P1 nunca quiere entrar (o se bloquea), P0 **nunca** podrá entrar.

```
void* proceso_0(void* arg) {
    // Código que no afecta datos compartidos
    while (turnoHilo != 0) {
        /* Espera su turno */
    }
    // Entra a la sección crítica
    saldo -= 500;
    // Cambio de turno
    turnoHilo = (turnoHilo + 1) % totalHilos;
    // Resto del código
    return NULL;
}

void* proceso_1(void* arg) {
    // Código que no afecta datos compartidos
    while (turnoHilo != 1) {
        /* Espera su turno */
    }
    // Entra a la sección crítica
    saldo -= 300;
    // Cambio de turno
    turnoHilo = (turnoHilo + 1) % totalHilos;
    // Resto del código
    return NULL;
}
```

Corrida en frío

Estado inicial:

saldo = 1000 y turnoHilo = 0

Proceso 0:

- como turnoHilo = 0, entra P0
- Realiza su sección crítica:
saldo = 1000 - 500 = 500
- aumenta turnoHilo = 1, P0 espera indefinidamente.

Proceso 1:

- inicialmente esperando
- como aumento turnoHilo = 1, entra P1
- ejecuta su sección crítica:
saldo = 500 - 300 = 200
- regresa turnoHilo = 0 y se mantiene esperando

Algoritmo de Dekker V2 - Flag de intención

Idea: Cada proceso señala que DESEA entrar usando `estadoHilo[ ]`. Si el otro no está dentro, entra.

- Si H0 quiere entrar, revisa si H1 está dentro.
- Si H1 está dentro, espera que salga.

Cabecera

```
// VARIABLES COMPARTIDAS
int estadoHilo[2] = {0, 0};
int saldo = 1000;
```

Problema

No garantiza la exclusión mutua:

Como existe un “hueco” de tiempo entre que el hilo comprueba la bandeja ajena y levanta la suya, el otro hilo puede aprovechar ese instante para entrar también.

Si el Hilo 1 realiza su comprobación justo en ese pequeño intervalo, también verá la bandera del hilo 0 en *false*. Como resultado, **ambos procesos pondrán sus banderas en *true* y ambos entrarán a la sección crítica a la vez.**

```
void* hilo_0(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    while (estadoHilo[1]) {
        /* Espera su turno */
    }
    estadoHilo[0] = 1;
    // Entra a la seccion critica
    saldo -= 500;
    // Resto del codigo
    estadoHilo[0] = 0;
    return NULL;
}

/*
 * Hilo 1: misma lógica, pero con su propia bandera.
 */
void* hilo_1(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    while (estadoHilo[0]) {
        /* Espera su turno */
    }
    estadoHilo[1] = 1;
    // Entra a la seccion critica
    saldo -= 300;
    // Resto del codigo
    estadoHilo[1] = 0;
    return NULL;
}
```

Corrida en frío

Estado inicial:

saldo = 1000, estadoHilo[0] = 0
(false) y estadoHilo[1] = 0 (false)

1. H0 ve que estadoHilo[1] = 0 y pasa el bucle.
 2. H1 ve que estadoHilo[0] = 0 aún y también pasa.
 3. H0 activa bandera estadoHilo[0] = 1
 4. H1 activa su bandera estadoHilo[1] = 1
 5. H0 ejecuta SC:
saldo = 1000 - 500 = 500
 6. **Colisión:** H1 también entra en SC: saldo = 1000 - 300 = **700**
- ¡Falla la exclusión mutua!

Algoritmo de Dekker V3 - Flag + Turn

Idea: Usa AMBAS variables. *flag* para intención, *turn* para romper empates.

Un proceso debe señalar su intención de entrar a la sección crítica antes de comprobar si el otro proceso está interesado.

Cabecera

```
// VARIABLES COMPARTIDAS
int estadoHilo[2] = {0, 0};
int turnoHilo = 0;
int saldo = 1000;
```

Problema

Aunque esta versión si garantiza la exclusión mutua, introduce un fallo crítico: el **interbloqueo** o deadlock.

El error ocurre cuando ambos procesos levantan sus banderas casi simultáneamente. Como ambos han declarado su intención de entrar antes de preguntar por el otro, **ambos** verán que el compañero también quiere entrar.

```
void* hilo_0(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[0] = 1;
    while (estadoHilo[1] && turnoHilo == 1) {
        /* Espera su turno */
    }
    // Entra a la sección crítica
    saldo -= 500;
    // Cambio de turno
    turnoHilo = 1;
    // Resto del código
    estadoHilo[0] = 0;
    return NULL;
}

void* hilo_1(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[1] = 1;
    while (estadoHilo[0] && turnoHilo == 0) {
        /* Espera su turno */
    }
    // Entra a la sección crítica
    saldo -= 300;
    // Cambio de turno
    turnoHilo = 0;
    // Resto del código
    estadoHilo[1] = 0;
    return NULL;
}
```

Corrida en frío

Estado inicial:

saldo = 1000, estadoHilo[0] = 0 (false) y estadoHilo[1] = 0 (false)

1. H0 declara intención de entrar, estadoHilo[0] = 1.
2. H1 declara intención también estadoHilo[1] = 1
3. H0 ve que H1 quiere entrar, entonces espera.
4. Pero H1 ve que H0 quiere entrar, entonces también espera.

5. ¡Interbloqueo (Deadlock)!

Algoritmo de Dekker V4 - Intención de retirada

Idea: Introduce un protocolo de “paso atrás”. En lugar de quedarse bloqueado esperando con la bandera levantada, el proceso detecta que el otro también quiere entrar y, baja su propia bandera, espera, y vuelve a intentarlo.

Cabecera

```
// VARIABLES COMPARTIDAS
int estadoHilo[2] = {0, 0};
int turnoHilo = 0;
int saldo = 1000;
```

```
void* hilo_1(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[1] = 1;
    while (estadoHilo[0]) {
        estadoHilo[1] = 0;
        // Cede turno al siguiente hilo
        while (turnoHilo != 1) {
            /* Espera su turno */
        }
        // Un tiempo de espera para que pase el siguiente
        estadoHilo[1] = 1;
    }
    // Entra a la seccion critica
    saldo -= 300;
    // Cambio de turno
    turnoHilo = 0;
    estadoHilo[1] = 0;
    // Resto del codigo
    return NULL;
}
```

```
void* hilo_0(void* arg) {
    /*
     * Código que no afecta datos compartidos
     */
    estadoHilo[0] = 1;
    while (estadoHilo[1]) {
        estadoHilo[0] = 0;
        // Cede turno al siguiente hilo
        while (turnoHilo != 0) {
            /* Espera su turno */
        }
        // Un tiempo de espera para que pase el siguiente
        estadoHilo[0] = 1;
    }
    // Entra a la seccion critica
    saldo -= 500;
    // Cambio de turno
    turnoHilo = 1;
    estadoHilo[0] = 0;
    // Resto del codigo
    return NULL;
}
```

Problema

Aunque elimina el interbloqueo, introduce un nuevo problema de vitalidad: el livelock o postergación indefinida.

Corrida en frío

Estado inicial:

saldo = 1000, estadoHilo[0] = 0 (false) y estadoHilo[1] = 0 (false)

1. H0 quiere entrar, estadoHilo[0] = 1
2. H1 quiere entrar también, estadoHilo[1] = 1
3. H0 y H1 entran en bucle de cortesía.
4. **Cortesía:** H0 baja su bandera, estadoHilo[0] = 0
5. **Cortesía:** H1 baja su bandera, estadoHilo[1] = 0.
6. H0 reintenta.
7. H1 reintenta simultáneamente.
8. **¡Bucle infinito (Livelock)!**

Algoritmo de Dekker V5 - Versión final

Idea: Introduce el concepto de **hilo favorecido**.

1. Un proceso marca su intención.
2. Si el otro proceso también tiene su bandera arriba, el proceso comprueba de quién es el turno
3. Si el turno del oponente, el proceso baja su propia bandera y espera activamente a que el turno cambie.
4. Una vez recupera el turno, levanta su bandera nuevamente y entra a la sección crítica.

```
// Protocolo de entrada: Indica que el hilo i quiere entrar a la sección crítica.
void entrada(int i) {
    int j = 1 - i;
    flag[i] = 1;

    while (flag[j]) {
        if (turn != i) {
            flag[i] = 0;
            while (turn != i) {
                /* Espera su turno */
            }
            flag[i] = 1;
        }
    }
}

// Protocolo de salida: Cede el turno al otro hilo y libera su intención.
void salida(int i) {
    turn = 1 - i;
    flag[i] = 0;
}
```

```
// Hilo 0: Retira 500 del saldo compartido.
void* hilo_0(void* arg) {
    // Código que no afecta datos compartidos
    entrada(0);

    // Entra a la sección crítica
    saldo -= 500;

    salida(0);
    // Resto del código
    return NULL;
}

// Hilo 1: Retira 300 del saldo compartido.
void* hilo_1(void* arg) {
    // Código que no afecta datos compartidos
    entrada(1);

    // Entra a la sección crítica
    saldo -= 300;

    salida(1);
    // Resto del código
    return NULL;
}
```

Cabecera

```
// VARIABLES COMPARTIDAS
int flag[2] = {0, 0};
int turn = 0;
int saldo = 1000;
```

Corrida en frío

Estado inicial:

saldo = 1000, flag[0] = 0, flag[1] = 0, turn = 0.

1. H0 quiere entrar, flag[0] = 1.
2. H1 quiere entrar también, flag[1] = 1.
3. H0 ve a H1, pero como turn == 0, insiste.
4. H1 ve a H0, pero como turn == 0, debe ceder.
5. **Cortesía:** H1 baja su bandera, flag[1] = 0, espera su turno.
6. H0 entra en SC, saldo = 1000 - 500 = 500. H1 se bloquea.
7. H0 termina su turno y lo cede a H1; baja su bandera flag[0] = 0
8. H1 sale de espera, sube bandera flag[1] = 1 y entra a SC.
9. H1 modifica saldo = 500 - 300 = 200.

¡Resultado correcto saldo = 200!

Algoritmo de Peierson

Supera la complejidad de la versión final de dekker al combiar de forma más sencilla el uso de **banderas de intención** y una **variable de turno**.

Idea: La lógica central es la “cortesía forzada”. Cuando un hilo P_i desea entrar en su sección crítica:

1. **Levanta su bandera** indicando que está listo ($flag[i] = true$).
2. **Cede inmediatamente el turno** al otro hilo P_j ($turn = j$).
3. **Espera** solo si el otro hilo también quiere entrar y es el turno del otro hilo ($while (flag[j] \&\& turn == j)$).

Esta estructura garantiza que, si ambos intentan entrar simultáneamente, la última asignación a la variable `turn` determinará quién espera, permitiendo que el otro progrese.

Cabecera

```
// VARIABLES COMPARTIDAS
bool flag[3] = {false, false};
int turn = 0;
int saldo = 1000;
```

Problemas

Espera activa: El hilo consume ciclos de CPU ejecutando un bucle mientras espera, lo que degrada la eficiencia del sistema.

Limitado a 2 procesos: En su forma básica, solo sirve para dos hilos.

```
// Hilo 0: Representa al primer proceso
void* hilo_0(void* arg) {
    // --- SECCIÓN DE ENTRADA ---
    flag[0] = true;
    turn = 1;
    while (flag[1] \&\& turn == 1);

    // --- SECCIÓN CRÍTICA ---
    saldo -= 500;
    printf("Hilo 0 ejecutó SC. Saldo: %d\n", saldo);

    // --- SECCIÓN DE SALIDA ---
    flag[0] = false;
    return NULL;
}

// Hilo 1: Representa al segundo proceso
void* hilo_1(void* arg) {
    // --- SECCIÓN DE ENTRADA ---
    flag[1] = true;
    turn = 0;
    while (flag[0] \&\& turn == 0);

    // --- SECCIÓN CRÍTICA ---
    saldo -= 300;
    printf("Hilo 1 ejecutó SC. Saldo: %d\n", saldo);

    // --- SECCIÓN DE SALIDA ---
    flag[1] = false;
    return NULL;
}
```


Algoritmo de Peterson

Corrida en Frío

Estado inicial:

saldo = 1000, flag[0] = 0, flag[1] = 0, turn = 0.

1. H0 quiere entrar, flag[0] = true
2. H1 quiere entrar, flag[1] = true
3. H0 cede el turno a H1, turn = 1
4. H1 cede el turno a H0 (sobreescribe) turn = 0.
5. H0 rompe el bucle y entra SC.
6. H1 se bloquea en el bucle.
7. H0 modifica saldo = 1000 - 500 = 500.
8. H0 termina y baja su bandera, flag[0] = false
9. H1 detecta flag[0] = false, sale del bucle y entra en SC
10. H1 modifica saldo = 500 - 300 = 200.

¡Resultado Correcto: saldo = 200!

```
// Hilo 0: Representa al primer proceso
void* hilo_0(void* arg) {
    // --- SECCIÓN DE ENTRADA ---
    flag[0] = true;
    turn = 1;
    while (flag[1] && turn == 1);

    // --- SECCIÓN CRÍTICA ---
    saldo -= 500;
    printf("Hilo 0 ejecutó SC. Saldo: %d\n", saldo);

    // --- SECCIÓN DE SALIDA ---
    flag[0] = false;
    return NULL;
}

// Hilo 1: Representa al segundo proceso
void* hilo_1(void* arg) {
    // --- SECCIÓN DE ENTRADA ---
    flag[1] = true;
    turn = 0;
    while (flag[0] && turn == 0);

    // --- SECCIÓN CRÍTICA ---
    saldo -= 300;
    printf("Hilo 1 ejecutó SC. Saldo: %d\n", saldo);

    // --- SECCIÓN DE SALIDA ---
    flag[1] = false;
    return NULL;
}
```

Algoritmo Panadería de Lamport²

Resuelve el problema de la exclusión mutua para un número n de procesos.

Cada cliente recibe un número al llegar y es atendido cuando su número es el más bajo. Se basa en la numeración incremental.

Toma de turno: antes de entrar en SC, cada hilo “toma un número.

Los hilos entran en orden creciente a sus tickets

Cabecera

```
// lamport_bakery_n_mutex.c
#include <stdio.h>
#include "mutex.h"

// Número de hilos
#define NUM_THREADS 5
// Cantidad que resta cada hilo
#define CANTIDAD_RESTA 100

// Variables compartidas
int choosing[NUM_THREADS];
int number[NUM_THREADS];
int saldo = 1000;

// Máximo de dos enteros
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Problemas

Eficiencia: Al igual que Peterson, utiliza espera activa, lo que consume ciclos CPU.

Complejidad: Difícil de implementar.

Multiprocesadores: Puede tener problemas de rendimiento o coherencia de memoria en arquitecturas multiprocesador modernas.

```
// Función que ejecuta cada hilo
void* hilo(void* arg) {
    int i = *(int*)arg; // índice del hilo
    free(arg); // liberar la memoria reservada en crear_n_procesos

    // Sección crítica
    entrada(i);
    // SC: restar una cantidad fija al saldo
    if (saldo >= CANTIDAD_RESTA) {
        printf("Hilo %d resta %d. Saldo antes: %d\n", i, CANTIDAD_RESTA, saldo);
        saldo -= CANTIDAD_RESTA;
        printf("Hilo %d termina SC. Saldo ahora: %d\n", i, saldo);
    } else {
        printf("Hilo %d no pudo restar (saldo insuficiente: %d)\n", i, saldo);
    }
    salida(i);
    return NULL;
}
```

```
// Protocolo de entrada para el hilo i
void entrada(int i) {
    int j;
    choosing[i] = 1;
    // Obtener el ticket máximo actual
    int max_number = 0;
    for (j = 0; j < NUM_THREADS; j++) {
        max_number = max(max_number, number[j]);
    }
    number[i] = max_number + 1;
    choosing[i] = 0;

    // Esperar a todos los hilos con ticket menor
    for (j = 0; j < NUM_THREADS; j++) {
        while (choosing[j]); // esperar a que j termine de elegir
        while (number[j] != 0 &&
            (number[j] < number[i] ||
            (number[j] == number[i] && j < i))) {
            // espera activa
        }
    }
}

// Protocolo de salida
void salida(int i) {
    number[i] = 0;
}
```

Creación n-hilos

```
// Crear los hilos usando la librería mutex extendida
ThreadArray hilos = crear_n_procesos(NUM_THREADS, hilo);

// Esperar a que terminen
esperar_n_procesos(hilos);

// Liberar recursos
liberar_n_procesos(hilos);
```

Algoritmo Panadería de Lampori

Corrida en Frío

Estado inicial:

NUM_THREADS = 3, saldo = 1000,
choosing = [0, 0, 0], number = [0, 0, 0]

1. Los tres Hilos H0, H1 y H2 anuncian elección; choosing = [1, 1, 1]
2. H0 obtiene ticket: calcula $\max(\text{number}[0, 0, 0]) = 0 \rightarrow \text{number}[0] = 1$.
3. H0 termina elección de ticket con choosing[0] = 0.
5. H1 y H2 obtienen sus tickets $\rightarrow$ choosing[0, 0, 0] y number[1, 2, 3]
6. Cada hilo ejecuta su bucle para esperar a los demás. H0 ticket = 1: $\text{number}[j=0, 1, 2] < 1 = \text{false} \rightarrow$ no espera, entra directo a SC.
7. H1 espera a H0 porque $\text{ticket}_0 = 1 < \text{ticket}_2 = 1$
8. H2 espera mientras H0 y H1 tengan ticket != 0.

9. H0 en SC modifica saldo $\text{saldo} -= 100 \rightarrow 900$

10. H0 sale SC $\text{number}[0] = 0$ y termina.

11. H1 ya tiene ticket libre y entra a SC.

12. H1 modifica saldo $\text{saldo} -= 100 \rightarrow 800$ y libera el ticket

13. Al terminar $\text{number}[0, 1, 2] = [0, 0, 3]$.

14. H2 ejecuta su SC y modifica saldo $\text{saldo} -= 100 \rightarrow 700$. H2 libera su ticket.

!Resultado final: saldo = 700;

```
// Protocolo de entrada para el hilo i
void entrada(int i) {
    int j;
    choosing[i] = 1;
    // Obtener el ticket máximo actual
    int max_number = 0;
    for (j = 0; j < NUM_THREADS; j++) {
        max_number = max(max_number, number[j]);
    }
    number[i] = max_number + 1;
    choosing[i] = 0;

    // Esperar a todos los hilos con ticket menor
    for (j = 0; j < NUM_THREADS; j++) {
        while (choosing[j]); // esperar a que j termine de elegir
        while (number[j] != 0 &&
            (number[j] < number[i] ||
             (number[j] == number[i] && j < i))) {
            // espera activa
        }
    }
}

// Protocolo de salida
void salida(int i) {
    number[i] = 0;
}
```

```
// Función que ejecuta cada hilo
void* hilo(void* arg) {
    int i = *(int*)arg; // índice del hilo
    free(arg); // liberar la memoria

    // Sección crítica
    entrada(i);
    // SC: restar una cantidad fija al saldo
    if (saldo >= CANTIDAD_RESTA) {
        printf("Hilo %d resta %d. Saldo antes\n", i, CANTIDAD_RESTA);
        saldo -= CANTIDAD_RESTA;
        printf("Hilo %d termina SC. Saldo ahora\n", i, saldo);
    } else {
        printf("Hilo %d no pudo restar (saldo insuficiente)\n", i);
    }
    salida(i);

    return NULL;
}
```


Soluciones de Hardware



Aprovechan capacidades específicas del procesador para simplificar la sincronización:

- **Inhabilitación de interrupciones:** En sistemas monoprocesador, desactivar las interrupciones impide que el proceso sea desalojado mientras está en su sección crítica.
- **Instrucciones atómicas:** Instrucciones de máquina especiales que realizan varias acciones (como leer y escribir) como una única unidad interrumpible.

Test and Set

Test&Set (TAS) es una instrucción atómica proporcionada por el hardware. Se basa en spinlocks.

Idea:

1. Se usa una variable compartida *lock* → lock=0: libre, lock=1: ocupado.
2. Para entrar a la SC, un hilo ejecuta TAS.
 - a. Si retorna 0 (lock está libre), el hilo adquiere el lock y entra a SC.
 - b. Si retorna 1 (lock ocupado), el hilo espera activamente y reintenta.
3. Al salir de la SC, lock = 0 para liberar.

Cabecera

```
// Variable compartida para el lock
// Usamos volatile para evitar optimizaciones del compilador
volatile int lock = 0;

// Recurso compartido
int saldo = 1000;

// Simulación de Test&Set atómico
// En hardware real, esto sería una instrucción única
// En GCC se puede usar __sync_lock_test_and_set
int TestAndSet(volatile int *target) {
    // En un sistema real, esto sería atómico
    // Usamos built-in de GCC para atomicidad real
    return __sync_lock_test_and_set(target, 1);
}
```

¿Resultado: saldo = 200!

Corrida en Frío

Estado inicial: saldo = 1000, lock = 0.

1. H0 ejecuta TAS y retorna 0.
2. H0 adquiere lock → 1 y entra a SC.
3. H1 ejecuta TAS(lock) = 1 y espera (spinning) el cambio.
4. H0 modifica saldo -= 500 = 500 y libera lock → 0.
5. H1 ejecuta TAS y retorna 0, lock → 1 y entra a SC.
6. H1 modifica saldo -= 300 = 200 y libera lock → 0.

```
void* hilo_0(void* arg) {
    // Sección no crítica

    // Protocolo de entrada (espera activa)
    while (TestAndSet(&lock) == 1) {
        // Spinning: espera activa hasta que el lock esté libre
    }

    // --- SECCIÓN CRÍTICA ---
    saldo -= 500;

    // Protocolo de salida
    lock = 0; // Liberar el lock
    return NULL;
}

void* hilo_1(void* arg) {
    // Sección no crítica

    // Protocolo de entrada
    while (TestAndSet(&lock) == 1) {
        // Espera activa
    }

    // --- SECCIÓN CRÍTICA ---
    saldo -= 300;

    // Protocolo de salida
    lock = 0;
    return NULL;
}
```

Operación atómica

```
int TestAndSet(int *target) {
    int old = *target; // Lee el valor actual
    *target = 1;        // Establece a 1
    return old;         // Devuelve el valor original
}
```


Compare And Swap

(CAS) es una instrucción más potente que TAS. Es la base de muchos algoritmos sin bloqueos (lock-free) y estructuras de datos concurrentes.

Idea:

1. Se usa una variable compartida *lock* → lock=0: libre, lock=1: ocupado.
2. Para adquirir el lock, un hilo intenta CAS:
 - a. Espera que lock sea 0.
 - b. Intenta cambiarlo a 1.
 - c. Si CAS retorna 0, logró adquirirlo.
 - d. Si retorna 1, otro hilo lo tiene y debe reintentar.
3. Para liberar, simplemente lock = 0.

Cabecera

```
// Variable compartida
volatile int lock = 0;
int saldo = 1000;

// Implementación de CAS usando built-in de GCC (atómico real)
int CompareAndSwap(volatile int *target, int expected, int new_value) {
    // __sync_val_compare_and_swap es atómico y retorna el valor original
    return __sync_val_compare_and_swap(target, expected, new_value);
}
```

¿Resultado: saldo = 200!

Corrida en Frío

Estado inicial: saldo = 1000, lock = 0. (para dos hilos simul.)

1. H0 y H1 ejecutan CAS (H0 primero).
2. Solo uno H0 ve old = 0,
3. H0 tiene éxito, H1 falla.
4. H0 entra a SC pero H1 reintenta hacer CAS.
5. H0 modifica saldo -= 500 = 500.
6. H1 falla otra vez lock = 1, retorna 1.
7. H0 libera lock = 0.
8. H1 vuelve a intentar y ahora si tiene éxito old = 0, 0==0 → lock = 1.
9. Ejecuta SC saldo -= 300 = 200 y libera lock = 0

```
void* hilo_0(void* arg) {
    // Protocolo de entrada con CAS
    while (1) {
        // Si lock es 0, intentamos cambiarlo a 1
        if (CompareAndSwap(&lock, 0, 1) == 0) {
            break; // Logró adquirir el lock
        }
        // Si no, sigue esperando (spinning)
    }

    // --- SECCIÓN CRÍTICA ---
    saldo -= 500;

    // Protocolo de salida
    lock = 0;
    return NULL;
}

void* hilo_1(void* arg) {
    while (1) {
        if (CompareAndSwap(&lock, 0, 1) == 0) {
            break;
        }
    }
    saldo -= 300;

    lock = 0;
    return NULL;
}
```

Operación atómica

```
int CompareAndSwap(int *target, int expected, int new_value) {
    int old = *target; // Lee el valor actual
    if (old == expected) { // Si coincide con lo esperado
        *target = new_value; // Actualiza al nuevo valor
    }
    return old; // Devuelve el valor original
}
```


TAS vs CAS

Característica	Test&Set (TAS)	Compare&Swap (CAS)
Operación básica	old = *target; *target = 1; return old;	if(*target == expected) *target = new; return old;
Valor de escritura	Fijo (siempre 1)	Variable (cualquier valor)
Información retornada	Solo valor anterior	Valor anterior (éxito)
Complejidad	Baja	Media
Flexibilidad	Baja	Alta
Caso de uso principal	Spinlocks básicos	Estructuras lock-free, colas, contadores.

Venías TAS

- Menor overhead por operación
 - TAS realiza solo lectura y escritura de 1.
 - CAS requiere comparación adicional.
 - En hardware, TAS puede ser ligeramente más rápido.
- Consume menos recursos de hardware
 - No necesita registro adicional para el valor esperado.
 - Implementación más simple en el procesador.
- Fácil de depurar
 - Menos posibilidades de error humano.
 - El comportamiento es más predecible

Venías CAS

- Alta flexibilidad.
- Permite operaciones RMW (Read-Modify-Write) complejas.
- Soporte para estructuras lock-free.
- Mayor eficiencia en ciertos escenarios
 - Permite optimizaciones como TTAS (Test-Test&Set)
 - En algunos patrones, CAS invalida menos caché
- Versatilidad para múltiples estados.

Desvenías CAS

- Alta flexibilidad.
- Permite operaciones RMW (Read-Modify-Write) complejas.
- Soporte para estructuras lock-free.
- Mayor eficiencia en ciertos escenarios
 - Permite optimizaciones como TTAS (Test-Test&Set)
 - En algunos patrones, CAS invalida menos caché
- Versatilidad para múltiples estados.

Desvenías TAS

- Poca flexibilidad.
- No permite operaciones lock-free complejas-
- Problemas de rendimiento en alta contención.
 - TAS escribe siempre, invalidando cachés constatemente.
 - Causa mucho tráfico en el bus de memoria.
- No proporciona información sobre el valor actual.

Algoritmos Clásicos



A continuación se presenta una explicación detallada de los algoritmos clásicos de sincronización, su funcionamiento e implementación.

Semáforos

¿Qué son?

Un semáforo es una variable especial (int o negativo) que soporta dos operaciones atómicas:

- **wait(s)** también llamada *P* o *down*:
 - Si $s > 0$, decrementa s y continúa.
 - Si $s == 0$, el hilo se bloquea hasta que s sea mayor que 0.
- **signal(s)**, también llamada *V* o *up*:
 - Incrementa s en 1. Si hay hilos bloqueados esperando, despierta a uno de ellos.

¿A qué problemas atienden?

Problema	Cómo ayuda el semáforo
Exclusión mutua	Semáforo binario (0/1) actúa como mutex
Sincronización productor-consumidor	Dos semáforos: empty (espacios libres) y full (elementos disponibles)
Control de acceso a recursos limitados	Semáforos contador inicializado con el número de recursos (ej: conexiones a BD)
Ordenamiento entre hilos	Un hilo espera señal de otro(semáforo = 0, otro hace signal)

```
#include "semaphore.h"

void init(semaphore *sem, int valor_inicial) {
    sem->value = valor_inicial;
    pthread_mutex_init(&sem->mutex, NULL);
    pthread_cond_init(&sem->cond, NULL);
}

void wait(semaphore *sem) {
    pthread_mutex_lock(&sem->mutex);
    while (sem->value == 0) {           // Si es cero, esperar
        pthread_cond_wait(&sem->cond, &sem->mutex);
    }
    sem->value--;                       // Decrementar atómicamente
    pthread_mutex_unlock(&sem->mutex);
}

void signal(semaphore *sem) {
    pthread_mutex_lock(&sem->mutex);
    sem->value++;                       // Incrementar
    pthread_cond_signal(&sem->cond);    // Despertar un hilo (si hay)
    pthread_mutex_unlock(&sem->mutex);
}

void destroy(semaphore *sem) {
    pthread_mutex_destroy(&sem->mutex);
    pthread_cond_destroy(&sem->cond);
}
```

Tipos de Semáforos

- **Binario:** valores solo 0 o 1 (equivalente a un mutex)
- **Contador:** puede tomar cualquier valor ≥ 0 .

Productor-Consumidor con buffer acotado

Idea:

- Uno o más **productores** generan datos y los colocan en un buffer compartido de tamaño fijo.
- Uno o más **consumidores** toman datos del buffer para procesarlos.

Las restricciones son:

- El productor no puede agregar si el buffer está **lleno**.
- El consumidor no puede tomar si el buffer está **vacío**.
- El acceso al buffer debe ser **mutuamente excluyente**.

Problemas

Condición de Carrera: Dos hilos modifican variables compartida (*in*, *out*, *count*) sin exclusión mutua, dando resultados inconsistentes.

Deadlock: Productor espera al consumidor y el consumidor espera al productor, ambos bloqueados indefinidamente.

Inanición: Un hilo nunca progresa porque otros siempre ocupan los recursos.

Espera activa: Usar while simple sin bloqueo consume CPU innecesariamente.

Productor más rápido que consumidor: El buffer se llena y el productor se bloquea; si no hay sincronización de lleno, se pierden datos o se sobrescribe.

Consumidor más rápido que productor: El buffer se vacía y el consumidor intenta leer datos no válidos.

Característica	Buffer acotado	Buffer no acotado
Tamaño	Fijo (máximo definido)	Teóricamente infinito (limitado por memoria disponible)
Sincronización necesaria	Tres semáforos: mutex, empty, full	Dos semáforos: mutex, full
Bloqueo del productor	Si, cuando el buffer está lleno	No, siempre puede producir (memoria disponible)
Riesgo de desbordamiento	No, el productor espera	Posible (memoria infinita no existe)
Uso típico	Sistemas con recursos limitados (cola de tareas, buffer de E/S)	Sistemas con memoria virtual grande, o cuando la producción es esporádica

Productor-Consumidor con buffer no acotado

Idea: El buffer crece de forma teóricamente ilimitada (por memoria). El productor **nunca se bloquea por falta de espacio**, solo necesita exclusión mutua para agregar elementos al buffer. El consumidor sigue necesitando sincronización para no consumir cuando el buffer está vacío.

Productor:

- Crea un nuevo nodo con *malloc*.
- Adquiere *sem_mutex* para proteger la estructura de la lista.
- Inserta el nodo al final.
- Libera *sem_mutex* y hace *signal(&sem_full)* para incrementar el contador de elementos disponibles.
- Nunca espera por *empty*, porque siempre hay espacio.

Consumidor:

- Espera en *wait(&sem_full)* para asegurar que haya al menos un elemento.
- Adquiere *sem_mutex*, extrae el primer nodo, libera memoria.
- Libera *sem_mutex* y luego consume el dato.

Problemas

Agotamiento de memoria: Si el productor produce más rápido de lo que el consumidor puede procesar, la lista crece sin límites y eventualmente el sistema se queda sin memoria.

Falta de control de flujo: No hay mecanismo para ralentizar al productor cuando el buffer es muy grande. En aplicaciones reales esto provoca caídas.

Característica	Buffer acotado	Buffer no acotado
Tamaño	Fijo (máximo definido)	Teóricamente infinito (limitado por memoria disponible)
Sincronización necesaria	Tres semáforos: mutex, empty, full	Dos semáforos: mutex, full
Bloqueo del productor	Si, cuando el buffer está lleno	No, siempre puede producir (memoria disponible)
Riesgo de desbordamiento	No, el productor espera	Posible (memoria infinita no existe)
Uso típico	Sistemas con recursos limitados (cola de tareas, buffer de E/S)	Sistemas con memoria virtual grande, o cuando la producción es esporádica

Productor-Consumidor con buffer acopiado

Cabecera

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "mutex.h"
#include "semaphore.h"

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

semaphore sem_mutex; // control de acceso al buffer (binario)
semaphore sem_empty; // espacios vacíos (inicial = BUFFER_SIZE)
semaphore sem_full; // elementos llenos (inicial = 0)
```

Main

```
int main() {
    printf("=== PRODUCTOR-CONSUMIDOR CON MI_SEM ===\n");
    init(&sem_mutex, 1); // mutex binario
    init(&sem_empty, BUFFER_SIZE);
    init(&sem_full, 0);

    ThreadPair hilos = crear_procesos(producer, consumidor);
    esperar_procesos(hilos);

    destroy(&sem_mutex);
    destroy(&sem_empty);
    destroy(&sem_full);

    printf("\nTerminado.\n");
    return 0;
}
```

Implementación

```
void* producer(void* arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        usleep(rand() % 200000);
        wait(&sem_empty); // espera un hueco
        wait(&sem_mutex); // protege buffer

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;

        signal(&sem_mutex);
        signal(&sem_full); // avisa que hay nuevo elemento
    }
    return NULL;
}

void* consumidor(void* arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        wait(&sem_full); // espera elemento
        wait(&sem_mutex); // protege buffer

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        signal(&sem_mutex);
        signal(&sem_empty); // avisa que hay un hueco

        usleep(rand() % 200000);
    }
    return NULL;
}
```

Corrida en frío

Estado inicial: Se producen 10 ítems.
vacio = 0 (Buffer lleno), lleno = 10,
mutex = 1

1. Productor se bloquea: Ejecuta wait(vacío) al querer insertar el ítem 11. Entonces espera.
2. El consumidor entra, ejecuta wait(lleno) y pasa a 0. Luego ejecuta wait(mutex) y pasa a 0.
3. El Consumidor extrae un ítem, ejecuta signal(mutex) y pasa a 1. Luego ejecuta signal(vacio) y vacio pasa a 1.
4. El productor se desbloquea.
5. El productor se reanuda, toma el mutex, inserta el ítem. Libera el mutex y ejecuta signal(lleno): lleno pasa a 10
6. Se restablece el equilibrio sin interbloqueo

Productor-Consumidor buffer no acopiado

Cabecera

Corrida en frío

```
// Buffer implementado como lista enlazada (sin tamaño fijo)
typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* head = NULL; // frente del buffer (de donde consume el consumidor)
Node* tail = NULL; // final del buffer (donde agrega el productor)

semaphore sem_mutex; // control de acceso al buffer
semaphore sem_full; // elementos disponibles (inicial = 0)
```

```
void* productor(void* arg) {
    for (int i = 0; i < 10; i++) {
        usleep(rand() % 200000);
        // Crear nuevo nodo
        Node* nuevo = (Node*)malloc(sizeof(Node));
        nuevo->data = i;
        nuevo->next = NULL;

        wait(&sem_mutex);
        // Añadir al final de la lista
        if (head == NULL) {
            head = tail = nuevo;
        } else {
            tail->next = nuevo;
            tail = nuevo;
        }
        printf("Productor: produce %d\n", i);
        signal(&sem_mutex);
        signal(&sem_full); // avisa que hay un elemento más
    }
    return NULL;
}
```

```
void* consumidor(void* arg) {
    for (int i = 0; i < 10; i++) {
        wait(&sem_full); // espera al menos un elemento
        wait(&sem_mutex);

        // Extraer del frente
        Node* nodo = head;
        int item = nodo->data;
        head = head->next;
        if (head == NULL) tail = NULL;
        free(nodo);
        printf("Consumidor: consume %d\n", item);

        signal(&sem_mutex);
        usleep(rand() % 200000);
    }
    return NULL;
}
```

Estado inicial: El consumidor aún no ha empezado (muy lento), lleno = 10, mutex = 1.

1. El productor produce el ítem 11 sin bloqueo previo.
 - a. Adquiere mutex con wait(&mutex) e inserta ítem 11 al final de la lista.
 - b. Luego libera el mutex signal(&mutex) y signal(&lleno); lleno = 11.
2. Consumidor empieza a ejecutarse: se quiere consumir un ítem
 - a. wait(&lleno) decrementa 11 → 10

b. wait(&mutex) adquiere mutex, extrae el ítem y head avanza

c. Libera el mutex signal(&mutex) y consume el ítem.

4. El buffer tiene 10 elementos, no hay hilos bloqueados, ambos pueden continuar.

Lectores y Escritores

Idea: Es un problema donde múltiples hilos comparten un recurso (ej. una base de datos) con diferentes modos de acceso:

- **Lectores:** solo leen el recurso. Pueden acceder simultáneamente (varios lectores)
- **Escritores:** Modifican el recurso. Deben tener acceso exclusivo (ni lectores ni otros escritores).

Variantes:

1. **Prioridad lectores:** Si hay lectores, los escritores esperan.

¡Nuestra Solución!

a. Problema: los escritores pueden morir de inanición si llegan muchos lectores.

2. **Prioridad escritores:** Los nuevos lectores esperan si hay escritores esperando.

a. Problema: Los lectores pueden morir de inanición.

3. **Justa:** Se atiende por orden de llegada (FIFO).

a. Problema: Más compleja de implementar.

Lógica del lector:

- Incrementa *lectores_activos* bajo mutex.
- Si es el primer lector, bloquea recurso (para que ningún escritor entre).
- Lee el recurso.
- Al salir, decrementa *lectores_activos*. Si es el último, libera recurso.

Variables

- *lectores_activos*: Cuántos lectores están leyendo actualmente.
- *mutex_lectores*: Protege el contador lectores activos
- *recurso*: semáforo binario que controla el acceso al recurso (1 = libre, 0 = ocupado)

Lógica del Escritor:

- Simplemente espera un *recurso* (que puede estar bloqueado por lectores o por otro escritor).
- Escribe.
- Libera *recurso*.

Problemas (prioridad lectores)

Inanición de escritores: Si llegan lectores continuamente, los escritores nunca escriben.

Condiciones de carrera: Sin *mutex_lectores*, dos lectores podrían modificar *lectores_activos* simultáneamente.

Deadlock: Si un escritor espera recurso y un lector espera algo que tiene el escritor tiene.

Sobrecarga de sincronización: Muchos semáforos pueden ralentizar el sistema.

Lectores y Escritores

Cabecera

```
// Recurso compartido
int datos_compartidos = 0;

// Variables de sincronización
semaphore mutex_lectores; // protege contador de lectores
semaphore recurso;        // controla acceso al recurso (para escritores)
int lectores_activos = 0;  // cuántos lectores están leyendo
```

Implementación

```
void* escritor(void* arg) {
    int id = *(int*)arg;
    free(arg);

    for (int i = 0; i < 3; i++) {
        usleep(rand() % 200000);

        // --- PROTOCOLO DE ENTRADA (ESCRITOR) ---
        wait(&recurso); // espera acceso exclusivo al recurso

        // --- SECCIÓN CRÍTICA (ESCRITURA) ---
        datos_compartidos++;
        printf("👉 Escritor %d: escribiendo datos = %d (lectores activos: %d)\n",
               id, datos_compartidos, lectores_activos);
        usleep(rand() % 100000); // simula tiempo de escritura

        // --- PROTOCOLO DE SALIDA (ESCRITOR) ---
        signal(&recurso);
    }
    return NULL;
}
```

Cabecera

Estado inicial: datos = 0, lectores_activos = 0, mutex_lectores = 1, recurso = 1; **Llega:** Lector0 → Escritor0 → Lector1

1. L0 entra: adquiere el mutex `wait(&mutex_lectores)` y lectores_activos = 1, bloquea el recurso `if(lectores_activos == 1)` `wait(&recurso)`, libera el mutex `signal(&mutex_lectores)` y entra en SC para leer dato = 0.

```
void* lector(void* arg) {
    int id = *(int*)arg;
    free(arg);

    for (int i = 0; i < 3; i++) {
        usleep(rand() % 100000);

        // --- PROTOCOLO DE ENTRADA (LECTOR) ---
        wait(&mutex_lectores);
        lectores_activos++;
        if (lectores_activos == 1) {
            wait(&recurso); // primer lector bloquea el recurso para escritores
        }
        signal(&mutex_lectores);

        // --- SECCIÓN CRÍTICA (LECTURA) ---
        printf("👈 Lector %d: leyendo datos = %d (lectores activos: %d)\n",
               id, datos_compartidos, lectores_activos);
        usleep(rand() % 50000); // simula tiempo de lectura

        // --- PROTOCOLO DE SALIDA (LECTOR) ---
        wait(&mutex_lectores);
        lectores_activos--;
        if (lectores_activos == 0) {
            signal(&recurso); // último lector libera el recurso
        }
        signal(&mutex_lectores);
    }
    return NULL;
}
```

2. E0 espera (`recurso=0`)
3. L1 entra: (misma operaciones de L0) `lectores_activos = 2`. Lee **datos = 0**.
4. L0 termina lectura y `lectores_activos = 1`.
5. L1 termina lectura → `lectores_activos = 0` y puede **liberar recurso** `signal(&recurso)`
6. E0 despierta, `wait(&recurso)` retorna
7. E0 escribe **datos = 1** → `signal(&recurso)`

Barbero dormilón

Problemas

Idea:

- Un barbero que atiende clientes.
- Una sala de espera con N sillas (buffer acotado).
- Clientes que llegan en momentos aleatorios.

Reglas:

- Si no hay clientes, el barbero duerme en su silla.
- Si llega un cliente y el barbero duerme, lo despierta.
- Si llega un cliente y hay sillas en espera, espera sentado.
- Si llega un cliente y todas las sillas están ocupadas, se va.

Variables:

- *mutex*: Protege el acceso a clientes_esperando.
- *cliente_listo*: Indica si hay clientes esperando. El barbero *wait()*, el cliente hace *signal()*
- *barbero_listo*: El cliente hace *wait()*, el barbero hace *signal()*

Clientes que esperan mucho tiempo: Si el barbero es muy lento y llegan muchos clientes, algunos pueden esperar indefinidamente en las sillas.

Prioridad implícita: Dependiendo de la implementación de los semáforos (FIFO o no), algunos clientes podrían ser siempre los primeros en ser atendidos.

Pérdida de clientes (por diseño): En sistemas reales puede representar pérdidas para el negocio.

Sin mecanismo de reintento: El cliente no puede intentar más tarde, aunque es poco realista para muchos sistemas.

```
#define NUM_SILLAS 3      // sillas en la sala de espera
#define NUM_CLIENTES 10  // clientes que llegan (para el ejemplo)

// Recursos compartidos
semaphore mutex;         // protege el contador de clientes_esperando
semaphore barbero_listo; // indica si el barbero está listo para atender (0 = durmiendo, 1 = disponible)
semaphore cliente_listo; // indica si hay clientes esperando (0 = no hay, >0 = hay)
int clientes_esperando = 0; // clientes en sillas de espera

// Estadísticas
int clientes_atendidos = 0;
int clientes_perdidos = 0;
```

Barbero dormilón

Estado inicial: clientes_esperando = 0, mutex = 1, cliente_listo = 0, barbero_listo = 0 : barbero durmiendo

1. C1 llega → despierta al barbero → C1 se sienta (clientes_esperando = 1)
2. Barbero atiende C1 → C1 sale
3. C2 llega → se sienta (clientes_esperando = 1) → barbero lo atiende
4. C3 llega mientras barbero atiende a C2 → se sienta (clientes_esperando = 1)
5. Barbero termina con C2 y luego atiende a C3
6. Si no hay más clientes el barbero vuelve a dormir.

```
void* barbero(void* arg) {
    while (1) {
        wait(&cliente_listo);    // espera a que llegue un cliente
        wait(&mutex);            // protege clientes_esperando

        clientes_esperando--;    // el cliente pasa de la silla de espera a la silla del barbero
        signal(&mutex);
        signal(&barbero_listo); // indica que el barbero está listo para atender

        // --- SECCIÓN CRÍTICA (ATENDER CLIENTE) ---
        usleep(rand() % 300000 + 200000); // atender toma tiempo

        printf("💇 Barbero: ✅ Cliente atendido\n");
    }
    return NULL;
}
```

```
void* cliente(void* arg) {
    int id = *(int*)arg;
    free(arg);

    wait(&mutex); // protege clientes_esperando

    if (clientes_esperando < NUM_SILLAS) {
        // Hay silla disponible
        clientes_esperando++;

        signal(&cliente_listo); // avisa al barbero que hay un cliente
        signal(&mutex);

        wait(&barbero_listo); // espera a que el barbero lo atienda

        // --- SECCIÓN CRÍTICA (SER ATENDIDO) ---
        usleep(rand() % 100000); // tiempo de atención percibido

        printf("👤 Cliente %d: 🚪 sale de la barbería\n", id);
    } else {
        // No hay sillas disponibles, el cliente se va
        clientes_perdidos++;
        signal(&mutex);
    }

    return NULL;
}
```

Filósofos comensales

Idea: Hay N filósofos (normalmente 5) sentados alrededor de una mesa circular.

- Cada filósofo tiene un plato de espaguetis. Entre cada par de filósofos hay un tenedor (o platillo).
- Para comer, un filósofo necesita dos tenedores: el de su izquierda y el de su derecha.
- Los filósofos alternan entre pensar (no usan tenedores) y comer (toman los dos tenedores, comen un rato, luego los sueltan).

Restricciones:

- Exclusión mutua: Dos filósofos vecinos no pueden comer simultáneamente porque comparten un tenedor.
- Sin deadlock: Que ningún filósofo se quede esperando indefinidamente (bloqueo total).
- Sin inanición: Que todos los filósofos puedan comer eventualmente.
- Concurrente: Si no hay conflictos, pueden comer varios filósofos a la vez (ej: los filósofos pares comen mientras los impares piensan).

```
#define N 5                // número de filósofos
#define PENSANDO 0
#define HAMBRIENTO 1
#define COMIENDO 2

semaphore tenedor[N];      // un semáforo por tenedor (1 = libre)
semaphore limite;          // permite solo N-1 filósofos a la vez
semaphore print_mutex;     // para ordenar la salida en pantalla
```

Problemas

Baja escalabilidad:

- **Número de semáforos:** Se necesita un semáforo por tenedor, más el semáforo de límite. Para N grande, la sobrecarga de mantenimiento puede ser alta.
- **Contención:** En sistemas de muchos filósofos, la contención sobre los tenedores y el semáforo *limite* crece, reduciendo el paralelismo real.
- **Cuello de botella:** La solución con *limite = N-1* fuerza que solo N-1 filósofos puedan competir, pero eso sigue siendo una limitación.

Semáforo:

- *tenedor[i]*: Protege el tenedor i (1 = libre, 0 = ocupado).
- *limite*: Limita el número de filósofos que pueden estar simultáneamente en la sección de toma de tenedores. Así se evita el deadlock (si N-1 filósofos ya tienen un tenedor, el último no podrá entrar y rompe el ciclo).
- *print_mutex*: Opcional: evita que las impresiones se mezclen (no es esencial para la exclusión mutua de los tenedores).

Filósofos comensales

```
void pensar(int id) {
    printf("Filósofo %d está PENSANDO\n", id);
    usleep(rand() % 800000);
}

void comer(int id) {
    printf("Filósofo %d está COMIENDO 🍴\n", id);
    usleep(rand() % 400000);
}

void* filosofo(void* arg) {
    int i = *(int*)arg;
    free(arg);

    while (1) {
        // ciclo infinito (filósofo activo)
        pensar(i);

        wait(&limite); // solo N-1 filósofos pueden pasar

        wait(&tenedor[i]); // toma tenedor izquierdo
        wait(&tenedor[(i+1) % N]); // toma tenedor derecho

        comer(i);

        signal(&tenedor[i]); // suelta izquierdo
        signal(&tenedor[(i+1) % N]); // suelta derecho

        signal(&limite); // libera cupo
    }
    return NULL;
}
```

Corrida en frío

Escenario: 3 filósofos (A, B, C). Tenedores inicialmente libres = 1. limite = 2

1. A entra (*limite* 2 → 1), toma tenedor izquierdo (0) y derecho (1). **Come**
2. B entra (limite 1 → 0), intenta tomar tenedor 1 (lo tiene A) → **se bloquea**.
3. C intenta entrar, pero limite = 0 → se bloquea en *wait(&limite)*.
4. A termina de comer, suelta tenedores 0 y 1, libera *limite* 0 → 1.
5. B se despierta, toma tenedor 1 y 2 → **come**.
6. C sigue bloqueado en *limite* = 0 porque B aún come (sin tenedores disponibles).
7. B termina, suelta tenedores y *limite* 1 → 2.
8. C entra, toma tenedor 2 y 0 → **come**.

Ejemplos

Modelo	Ejemplo
Productor-Consumidor con buffer acotado	Cola de impresión en red: cuando un usuario envia un documento, el sistema intenta agregarlo a la cola compartida. Si la cola está llena, el usuario se bloquea (o recibe un mensaje de error). Cuando la impresora termina un trabajom toma el siguiente de la cola. Si la cola estpa vacia, la impresora entra en reposo.
Productor-Consumidor con buffer no acotado	Servidor web que acepta todas las peticiones sin rechazo, encolándolas en memoria dinámica: Cada peticion se añade a una lista enlazada. El servidor no rechaza ninguna por capacidad. Si el ritmo de llegada supera el de procesamiento, la cola crece indefinidamente, lo que puede llevar a una denegaciónde servicio por consumo de memoria.
Lectores-Escritores	Sistema de información de vuelos de un aeropuerto: Múltiples lectores pueden acceder a la vez sin conflictos, porque solo consultan. Cuando un escritor necesita modificar, debe esperar a que no haya ningun lector activo. • Lectores: Las pantallas de información que muestran horarios de llegada y salida. Pueden leer simultáneamente la base de datos de vuelos. • Escritores: Personal de control que actualiza la base de datos (retrasos, etc.)
Barbero dormilón	Centro de atención al cliente de una empresa de telecom: Si no hay llamadas el agente está inactivo. Cuando llega una llamada: si el agente está libre la atiende inmediatamente, si el agente está ocupado y hay silals libres, la llamada se pone es espera, si todas las sillas están ocupadas, la llamada se pierde (se rechaza o se desvía a buzón).
Filósofos comensales	Sistema de reservas de vuelos donde cada transacción bloquea un asiento y un vuelo: Para realizar una reserva, una transacción necesita bloquear el asiento específico y el vuelo al que pertenece. Si varias transacciones intentan bloquear los recursos en órdenes diferentes, puede ocurrir un deadlock. La solución típica es forzar un orden global de adquisición como un semáforo.

Interbloqueos



Es un estado en un sistema operativo en el que dos o más procesos permanecen atascados para siempre porque cada uno espera un recurso que posee otro. Solo ocurre cuando existen cuatro condiciones: exclusión mutua, espera y retención, sin preempción y espera circular.

El bloqueo puede gestionarse mediante prevención, evitación (algoritmo del banquero) o detección y recuperación.

Diagrama resumen

Método / Concepto	Definición	Principios Fundamentales	Funcionamiento
Prevención (Prevention)	Diseñar el sistema de modo que sea teóricamente imposible que ocurra un interbloqueo	Atacar y eliminar al menos una de las cuatro condiciones de Coffman (Exclusión mutua, Retención y espera, Sin desalojo y Espera circular).	Exclusión Mutua: Compartir recursos. Retención y espera: Exigir que los procesos soliciten todos sus recursos al inicio. Sin desalojo: Si un proceso pide un recurso y se le niega, debe liberar los que ya tiene. Espera circular: Establecer un orden lineal estricto para la solicitud de recursos.
Detección (Detection)	Permitir que el interbloqueo ocurra de manera libre pero emplea algoritmos periódicos para identificarlo	Monitoriar el estado del sistema mediante estructuras de datos para descubrir si se ha formado un ciclo de espera.	El sistema debe ejecutar el algoritmo de detección periódicamente cuando baja el rendimiento de la CPU. Requiere el uso de un Grafo de Asignación de Recursos (RAG) o matrices de asignación/petición. (algoritmo del banquero en modo detección).
Recuperación (Recovery)	Acciones que toma el sistema una vez que el algoritmo de detección ha confirmado la existencia de un interbloqueo	Romper el ciclo de interbloqueo eliminando procesos o quitándoles recursos de manera forzada	Se aplica inmediatamente después de la Detección. Métodos principales: Terminación de procesos: Abortar todos los procesos implicados o uno por uno hasta romper el ciclo. Desalojo de recursos: Quitar un recurso a un proceso (haciendo <i>rollback</i> a un estado seguro anterior) y dárselo a otro.
Inanición (Starvation)	No es un método, sino un problema donde un proceso queda postergado indefinidamente mientras otros avanzan	Ocurre por una asignación de recursos injusta o como efecto secundario de la recuperación (ej: elegir al proceso como víctima de desalojo).	Un proceso de baja prioridad solicita un recurso que constantemente se le asigna a procesos de mayor prioridad. Se soluciona con técnicas de envejecimiento (aging) , aumentando la prioridad del proceso conforme pasa el tiempo.

Algoritmo del Banquero

Escenario: Sistema con 5 hilos (H0 - H4) y 3 Recursos (A, B, C)

- A = 10, B = 5, C = 7
- Disponible (Work): [3, 3, 2]

Proceso / Hilo	Asignación (Curr)	Máximo (Max)	Necesidad (Need)
H0	0 - 1 - 0	7 - 5 - 3	7 - 4 - 3
H1	2 - 0 - 0	3 - 2 - 2	1 - 2 - 2
H2	3 - 0 - 2	9 - 0 - 2	6 - 0 - 0
H3	2 - 1 - 1	2 - 2 - 2	0 - 1 - 1
H4	0 - 0 - 2	4 - 3 - 3	4 - 3 - 1

Corrida en frío

Secuencia:

1. **H1:** Necesita [1, 2, 2] <= Disponible.
Ejecuta y libera [2, 0, 0] → nuevo disp: [5, 3, 2].
2. **H3:** Necesita [0, 1, 1] <= Disp → Libera [2, 1, 1] → Disponible = [7, 4, 5]
3. **H4:** Necesita [4, 3, 1] <= Disp → libera [0, 0, 2] → disp = [7, 4, 5]
4. **H2:** Libera [3, 0, 2] → Disp = [10, 4, 7]
5. **H0:** Libera [0, 1, 0]. Disp. final = [10, 5, 7]

Resultado

El sistema está en un estado seguro con la secuencia: H1 → H3 → H4 → H2 → H0

Muchas gracias